

The Rabin–Karp Algorithm

1 Introduction

Definition

The Rabin–Karp algorithm is a string-matching algorithm that uses **hash values** to compare the pattern with substrings of the text.

Suppose:

$$T[1 : n] = \text{text}, \quad P[1 : m] = \text{pattern}, \quad m \leq n.$$

The basic idea is:

- compute a numeric value for the pattern,
- compute numeric values for each text window of length m ,
- compare these numeric values first,
- verify characters only when the values match.

Why this helps

Direct comparison at every shift may take up to m time per shift. Rabin–Karp avoids this by using a **rolling hash**, so the next window value can be updated quickly.

2 Main Idea of the Algorithm

If the alphabet is digits, for example

$$\Sigma = \{0, 1, 2, \dots, 9\},$$

then each string can be viewed as a number.

Example:

$$31415 \longrightarrow 31415.$$

Let:

$$p = \text{hash value of the pattern } P,$$

and for each shift s ,

$$t_s = \text{hash value of } T[s + 1 : s + m].$$

Then:

$$t_s = p$$

suggests that the current text window may match the pattern.

Important

Hash equality does **not always** guarantee real equality. Sometimes two different strings have the same hash value. This is called a **spurious hit**.

3 Hash Computation

3.1 Pattern Hash

The pattern hash is computed using Horner's rule:

$$p = P[m] + 10(P[m-1] + 10(P[m-2] + \cdots + 10P[1])).$$

This takes:

$$\boxed{\Theta(m)}$$

time.

3.2 First Window Hash

Similarly, the first text window

$$T[1 : m]$$

is converted into

$$t_0.$$

Initial Hashes

$$p = \text{hash of } P[1 : m], \quad t_0 = \text{hash of } T[1 : m].$$

4 Rolling Hash Formula

The most important part of Rabin-Karp is the rolling update.

If we already know t_s , then we compute t_{s+1} without recomputing the whole window.

Rolling Hash Update

For decimal digits,

$$t_{s+1} = 10(t_s - 10^{m-1}T[s+1]) + T[s+m+1].$$

Meaning

- remove the leftmost digit,
- shift remaining digits to the left,
- add the new rightmost digit.

So the next hash is found in:

$$\boxed{\Theta(1)}$$

time.

5 Why Modular Arithmetic is Used

If m is large, then the numbers become too large.

To fix this, Rabin–Karp computes everything modulo a prime q .

Modular Version

Instead of storing p and t_s , we store:

$$p \bmod q \quad \text{and} \quad t_s \bmod q.$$

For a general radix d , define

$$h \equiv d^{m-1} \pmod{q}.$$

Then the rolling formula becomes:

General Modular Formula

$$t_{s+1} = (d(t_s - T[s+1]h) + T[s+m+1]) \pmod{q}.$$

6 Step-by-Step Solved Example 1

Problem

Let

$$P = 31415, \quad m = 5,$$

and suppose the current text window is also

$$31415.$$

Now the next text window is formed by removing the first digit 3 and adding the new digit 2.
Find the next hash value.

Solution

Current window:

$$t_s = 31415.$$

Outgoing digit:

$$T[s+1] = 3.$$

Incoming digit:

$$T[s+6] = 2.$$

Using the rolling formula:

$$t_{s+1} = 10(t_s - 10^{m-1}T[s+1]) + T[s+m+1].$$

Since $m = 5$,

$$10^{m-1} = 10^4 = 10000.$$

So:

$$t_{s+1} = 10(31415 - 10000 \cdot 3) + 2.$$

Now solve step by step:

$$\begin{aligned} t_{s+1} &= 10(31415 - 30000) + 2 \\ &= 10(1415) + 2 \\ &= 14150 + 2 \\ &= \boxed{14152}. \end{aligned}$$

Answer

The next window hash is:

$$\boxed{14152}.$$

7 Step-by-Step Solved Example 2 with Modulo

Problem

Let

$$P = 31415, \quad q = 13.$$

Find the pattern hash modulo 13. Then find the next rolling hash modulo 13 when

$$t_s = 31415, \quad m = 5, \quad d = 10,$$

the outgoing digit is 3, and the incoming digit is 2.

Solution Part A: Pattern Hash Modulo 13

We calculate:

$$31415 \bmod 13.$$

Divide:

$$13 \times 2416 = 31408.$$

So:

$$31415 - 31408 = 7.$$

Hence:

$$\boxed{31415 \equiv 7 \pmod{13}}.$$

So the pattern hash is:

$$\boxed{p = 7}.$$

Solution Part B: Next Window Hash Modulo 13

We use the modular rolling formula:

$$t_{s+1} = (10(t_s - T[s+1]h) + T[s+6]) \pmod{13}.$$

First compute:

$$h = 10^{m-1} \pmod{13} = 10^4 \pmod{13}.$$

Now:

$$10^2 = 100 \equiv 9 \pmod{13},$$

so

$$10^4 \equiv 9^2 = 81 \equiv 3 \pmod{13}.$$

Thus:

$$h = 3.$$

Also:

$$t_s \equiv 7 \pmod{13}.$$

Now substitute:

$$t_{s+1} = (10(7 - 3 \cdot 3) + 2) \pmod{13}.$$

Simplify:

$$\begin{aligned} &= (10(7 - 9) + 2) \pmod{13} \\ &= (10(-2) + 2) \pmod{13} \\ &= (-20 + 2) \pmod{13} \\ &= -18 \pmod{13}. \end{aligned}$$

Since

$$-18 + 26 = 8,$$

we get

$$t_{s+1} \equiv 8 \pmod{13}.$$

Answer

$$p = 7, \quad t_{s+1} = 8 \text{ modulo } 13.$$

8 Spurious Hit Example

Idea

Suppose the pattern hash is

$$p = 7 \pmod{13}.$$

If a text window also has hash

$$t_s = 7 \pmod{13},$$

then we call it a **hit**.

But there are two possibilities:

- it is a **true match**, or
- it is a **spurious hit**.

Example

Pattern:

$$P = 31415.$$

Suppose one text window also gives:

$$t_s = 7 \pmod{13}.$$

Now:

- if the actual substring is 31415, then it is a real match;
- if the actual substring is different but still has remainder 7, then it is a spurious hit.

Conclusion

Hash equality means:

possible match

not

certain match.

So direct character comparison is still necessary after a hit.

9 Algorithm Steps Properly Explained

Rabin–Karp Procedure

```
RABIN-KARP-MATCHER(T, P, d, q)
1  n = T.length
2  m = P.length
3  h = d^(m-1) mod q
4  p = 0
5  t0 = 0
6  for i = 1 to m
7      p = (d*p + P[i]) mod q
8      t0 = (d*t0 + T[i]) mod q
9  for s = 0 to n - m
10     if p == ts
11         if P[1:m] == T[s+1:s+m]
12             print "Pattern occurs with shift", s
13     if s < n - m
14         ts+1 = (d(ts - T[s+1]h) + T[s+m+1]) mod q
```

Explanation of Each Part

- Lines 1–2: read lengths of text and pattern.
- Line 3: compute the value of the highest place $h = d^{m-1} \pmod{q}$.
- Lines 4–8: compute the pattern hash p and first window hash t_0 .

- Line 9: slide through all valid shifts.
- Line 10: compare hashes.
- Line 11: verify actual strings if hashes match.
- Line 14: compute next window hash in constant time.

10 Full Solved Matching Example

Problem

Use Rabin–Karp to search for

$$P = 26$$

in

$$T = 3141592653589793$$

using

$$d = 10, \quad q = 11.$$

Show the complete matching process.

Given Data

Pattern:

$$P = 26, \quad m = 2$$

Text:

$$T = 3141592653589793, \quad n = 16$$

So the possible shifts are

$$s = 0, 1, 2, \dots, 14.$$

Step 1: Compute h

In Rabin–Karp,

$$h = d^{m-1} \pmod{q}.$$

Since $d = 10$, $m = 2$, and $q = 11$,

$$h = 10^{2-1} \pmod{11} = 10^1 \pmod{11} = 10.$$

So,

$$\boxed{h = 10.}$$

Step 2: Compute the Pattern Hash

The pattern is

$$P = 26.$$

Its hash is

$$p = 26 \pmod{11}.$$

Now,

$$26 = 22 + 4.$$

So,

$$26 \equiv 4 \pmod{11}.$$

Hence,

$$\boxed{p = 4.}$$

Step 3: Compute the First Text Window Hash

The first text window is

$$T[1 : 2] = 31.$$

So,

$$t_0 = 31 \pmod{11}.$$

Since

$$31 = 22 + 9,$$

we get

$$31 \equiv 9 \pmod{11}.$$

Thus,

$$\boxed{t_0 = 9.}$$

Since

$$t_0 \neq p,$$

shift $s = 0$ is rejected.

Step 4: Rolling Hash Computation for All Shifts

The rolling formula is

$$t_{s+1} = (d(t_s - T[s+1]h) + T[s+m+1]) \pmod{q}.$$

Since

$$d = 10, \quad h = 10, \quad q = 11,$$

this becomes

$$t_{s+1} = (10(t_s - 10T[s+1]) + T[s+3]) \pmod{11}.$$

We now compute each shift one by one.

Shift $s = 0$

Window:

$$31$$

Hash:

$$t_0 = 9.$$

Compare with pattern hash:

$$9 \neq 4.$$

So shift 0 is rejected.

Compute t_1

Old window: 31, new window: 14

Using the rolling formula:

$$t_1 = (10(t_0 - 3 \cdot 10) + 4) \pmod{11}.$$

Substitute $t_0 = 9$:

$$\begin{aligned} t_1 &= (10(9 - 30) + 4) \pmod{11} \\ &= (10(-21) + 4) \pmod{11} \\ &= -206 \pmod{11}. \end{aligned}$$

Now

$$-206 = -220 + 14,$$

so

$$-206 \equiv 14 \equiv 3 \pmod{11}.$$

Thus,

$$\boxed{t_1 = 3.}$$

Shift $s = 1$

Window:

$$14$$

Hash:

$$t_1 = 3.$$

Compare:

$$3 \neq 4.$$

So shift 1 is rejected.

Compute t_2

Old window: 14, new window: 41

$$t_2 = (10(t_1 - 1 \cdot 10) + 1) \pmod{11}.$$

Substitute $t_1 = 3$:

$$\begin{aligned} t_2 &= (10(3 - 10) + 1) \pmod{11} \\ &= (10(-7) + 1) \pmod{11} \\ &= -69 \pmod{11}. \end{aligned}$$

Since

$$-69 = -77 + 8,$$

we get

$$\boxed{t_2 = 8.}$$

Shift $s = 2$

Window:

$$41$$

Hash:

$$t_2 = 8.$$

Compare:

$$8 \neq 4.$$

So shift 2 is rejected.

Compute t_3

Old window: 41, new window: 15

$$t_3 = (10(t_2 - 4 \cdot 10) + 5) \pmod{11}.$$

Substitute $t_2 = 8$:

$$\begin{aligned} t_3 &= (10(8 - 40) + 5) \pmod{11} \\ &= (10(-32) + 5) \pmod{11} \\ &= -315 \pmod{11}. \end{aligned}$$

Since

$$-315 = -319 + 4,$$

we get

$$\boxed{t_3 = 4.}$$

Shift $s = 3$

Window:

$$15$$

Hash:

$$t_3 = 4.$$

Now

$$t_3 = p.$$

So this is a **hit**. Now we verify the actual strings:

$$15 \neq 26.$$

So shift 3 is a **spurious hit**.

Compute t_4

Old window: 15, new window: 59

$$t_4 = (10(t_3 - 1 \cdot 10) + 9) \pmod{11}.$$

Substitute $t_3 = 4$:

$$\begin{aligned} t_4 &= (10(4 - 10) + 9) \pmod{11} \\ &= (10(-6) + 9) \pmod{11} \\ &= -51 \pmod{11}. \end{aligned}$$

Since

$$-51 = -55 + 4,$$

we get

$$\boxed{t_4 = 4.}$$

Shift $s = 4$

Window:

$$59$$

Hash:

$$t_4 = 4.$$

Again,

$$t_4 = p.$$

So this is another hit.

Verify:

$$59 \neq 26.$$

Hence shift 4 is also a **spurious hit**.

Compute t_5

Old window: 59, new window: 92

$$t_5 = (10(t_4 - 5 \cdot 10) + 2) \pmod{11}.$$

Substitute $t_4 = 4$:

$$\begin{aligned} t_5 &= (10(4 - 50) + 2) \pmod{11} \\ &= (10(-46) + 2) \pmod{11} \\ &= -458 \pmod{11}. \end{aligned}$$

Since

$$-458 = -462 + 4,$$

we get

$$\boxed{t_5 = 4.}$$

Shift $s = 5$

Window:

$$92$$

Hash:

$$t_5 = 4.$$

So this is a hit.

Verify:

$$92 \neq 26.$$

Thus shift 5 is also a **spurious hit**.

Compute t_6

Old window: 92, new window: 26

$$t_6 = (10(t_5 - 9 \cdot 10) + 6) \pmod{11}.$$

Substitute $t_5 = 4$:

$$\begin{aligned} t_6 &= (10(4 - 90) + 6) \pmod{11} \\ &= (10(-86) + 6) \pmod{11} \\ &= -854 \pmod{11}. \end{aligned}$$

Since

$$-854 = -858 + 4,$$

we get

$$\boxed{t_6 = 4.}$$

Shift $s = 6$

Window:

$$26$$

Hash:

$$t_6 = 4.$$

So this is a hit.

Now verify actual strings:

$$26 = 26.$$

Therefore, shift 6 is a **valid match**.

First Real Match

The pattern occurs at

$$\boxed{s = 6.}$$

Continue Remaining Shifts

We continue the same process for the rest of the windows.

Shift s	Window	Hash mod 11	Result
0	31	9	reject
1	14	3	reject
2	41	8	reject
3	15	4	hit, but spurious
4	59	4	hit, but spurious
5	92	4	hit, but spurious
6	26	4	valid match
7	65	10	reject
8	53	9	reject
9	35	2	reject
10	58	3	reject
11	89	1	reject
12	97	9	reject
13	79	2	reject
14	93	5	reject

Verification of Remaining Hashes

For completeness, the remaining window remainders are:

$$\begin{aligned}65 &\equiv 10 \pmod{11}, & 53 &\equiv 9 \pmod{11}, & 35 &\equiv 2 \pmod{11}, \\58 &\equiv 3 \pmod{11}, & 89 &\equiv 1 \pmod{11}, & 97 &\equiv 9 \pmod{11}, \\79 &\equiv 2 \pmod{11}, & 93 &\equiv 5 \pmod{11}.\end{aligned}$$

None of these equals the pattern hash 4, so no more candidate matches occur.

Final Conclusion

The Rabin–Karp algorithm finds:

- **spurious hits** at shifts

$$3, 4, 5,$$

because their hash values equal the pattern hash but the actual strings do not match;

- one **real match** at shift

$$\boxed{6}.$$

Final Answer

For

$$P = 26, \quad T = 3141592653589793, \quad d = 10, \quad q = 11,$$

the Rabin–Karp algorithm reports:

Pattern occurs with shift 6.

It also encounters 3 spurious hits, at shifts

3, 4, 5.

11 Running Time Analysis

11.1 Preprocessing Time

We compute:

- h ,
- pattern hash p ,
- first window hash t_0 .

So preprocessing time is:

$$\Theta(m).$$

11.2 Matching Time

There are

$$n - m + 1$$

shifts.

At each shift:

- hash comparison takes constant time,
- rolling update takes constant time,
- but exact verification after a hit may take m time.

So worst-case matching time is:

$$\Theta((n - m + 1)m).$$

11.3 Worst Case Example

If

$$P = a^m \quad \text{and} \quad T = a^n,$$

then every shift becomes a hit and must be fully verified.

Thus total time is:

$$\Theta((n - m + 1)m).$$

11.4 Expected Case

If spurious hits are rare and actual matches are few, then the algorithm behaves much better in practice, often near:

$$\mathcal{O}(n + m).$$

12 Advantages and Disadvantages

Point	Description
Main strength	Fast filtering using hash values
Update cost	Next window hash in $\Theta(1)$ time
Preprocessing	$\Theta(m)$
Worst case	$\Theta((n - m + 1)m)$
Main weakness	Spurious hits due to collisions
Practical benefit	Often much faster than naive matching in average cases

13 Final Summary

One-Box Summary

Rabin–Karp compares hash values instead of comparing full strings first
Pattern hash p and window hash t_s are computed
Rolling hash update: $t_{s+1} = (d(t_s - T[s + 1]h) + T[s + m + 1]) \bmod q$
If $t_s = p$, verify characters directly
Spurious hit: same hash but different strings
Preprocessing time: $\Theta(m)$
Worst-case matching time: $\Theta((n - m + 1)m)$
Expected practical performance is often much better